



Group 09

Dysarthria and Non-Dysarthria Speech Classification

Vo Tran Hien - Pham Minh Duc - Vu Mai Thuy - Nguyen Dang Huu Thinh

University of Science, Viet Nam National University, HCMC
Faculty of Information Technology

March 21, 2025

History

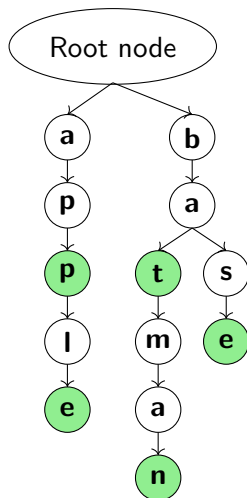
- ★ The **Trie** data structure was first introduced by **René de la Briandais** in 1959 as an efficient method for storing and retrieving strings in a lexicon
- ★ It was later independently developed by **Edward Fredkin** in 1960.



Figure: Edward Fredkin

Definition

A **Trie** (also known as a **prefix tree**) is a tree-like data structure used to store a dynamic set or associative array where the keys are usually strings.



Purpose

The primary purpose of a Trie is to efficiently handle dictionary operations like searching, insertion, and deletion of words. It is widely used in applications like autocomplete, spell checking, IP routing, and DNA sequencing.

Main Data Structure

- ★ A Trie consists of nodes, where each node represents a character of a stored key.
- ★ Each node contains an array (or hash map) of child nodes, representing possible next characters in stored keys.
- ★ A boolean flag is often used at each node to indicate whether it marks the end of a valid word.

Insertion

- ★ Starting from the root, check if the current character exists as a child node. If not, create a new node.
- ★ Move to the child node and repeat for the next character.
- ★ Mark the last node as the end of a word.

Searching

- ★ Start at the root and follow the path of characters in the query string.
- ★ If a path exists for the entire query string, check if the final node is marked as the end of a word.
- ★ Return true if the word is found, otherwise return false.

Deletion

- ★ Traverse the Trie following the word's characters.
- ★ Unmark the last character node as the end of a word.
- ★ If the node has no other children, recursively remove it to save space.

Node definition

```
5 struct TrieNode{  
6     map <char,TrieNode*> children;  
7     bool EndofWord;  
8  
9     TrieNode(){  
10         EndofWord = false;  
11     }  
12 };
```

Insert: $O(L)$

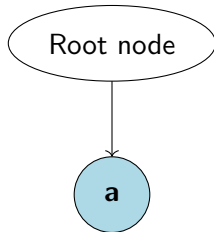
```
14 void insert(TrieNode* &proot, const string& word){
15     if (!proot) return;
16     TrieNode* temp = proot;
17
18     // Traverse each character of the word
19     for (char ch : word){
20         // If the character is not present, create a new TrieNode
21         if(temp->children.find(ch)==temp->children.end())
22             temp->children[ch] = new TrieNode;
23         temp = temp->children[ch];
24     }
25     // Mark the last node as the end of a word
26     temp->EndofWord=true;
27 }
```

Visualize

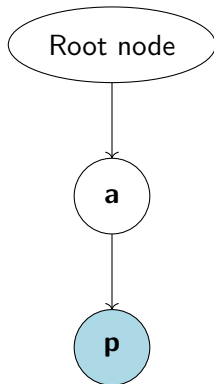
Assume we need to insert 5 words:

- ★ app
- ★ batman
- ★ bat
- ★ base

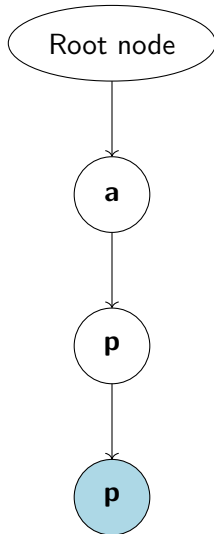
Insert "apple"



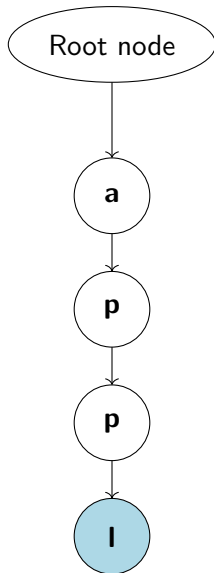
Insert "apple"



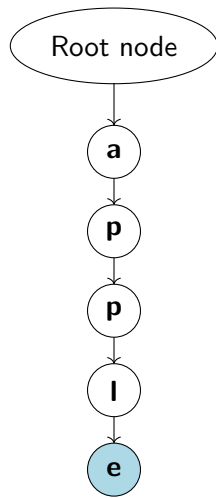
Insert "apple"



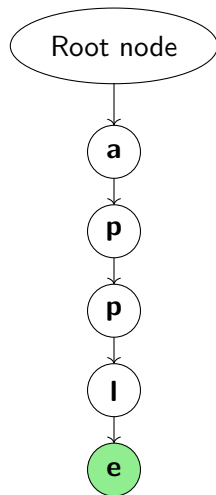
Insert "apple"



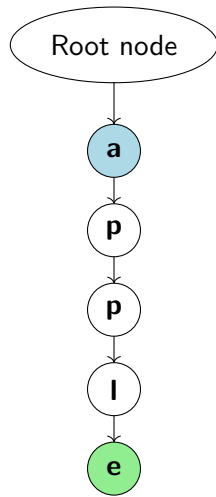
Insert "apple"



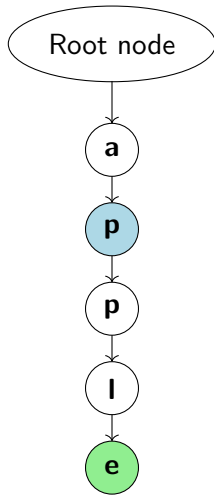
Insert "apple"



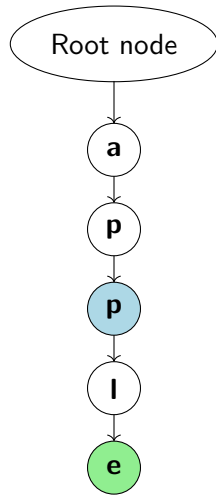
Insert "app"



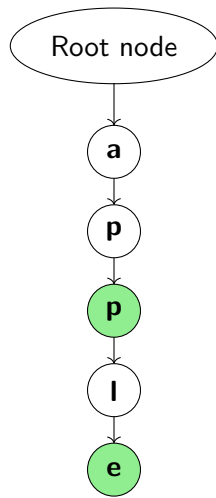
Insert "app"



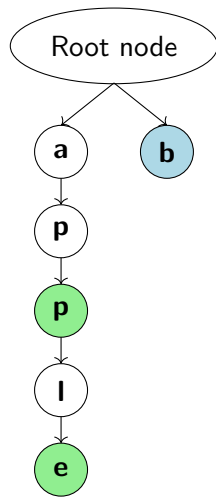
Insert "app"



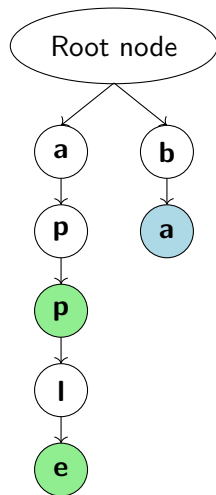
Insert "app"



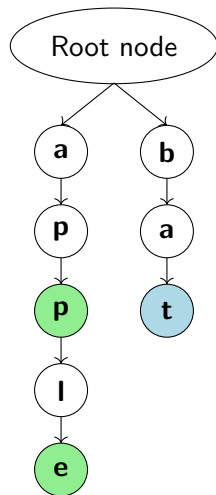
Insert "bat"



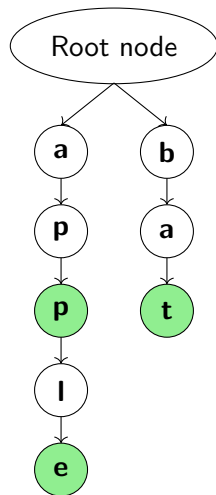
Insert "bat"



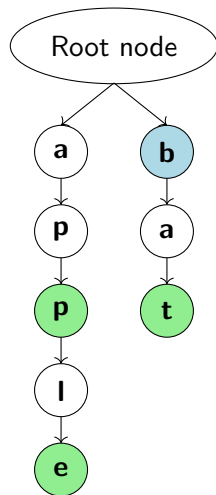
Insert "bat"



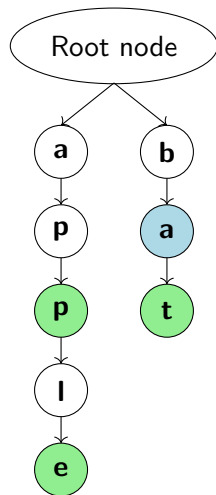
Insert "bat"



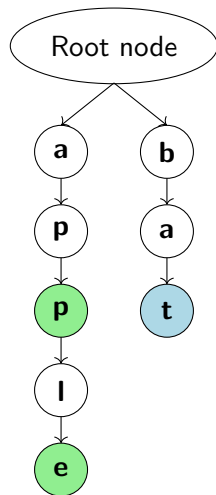
Insert "batman"



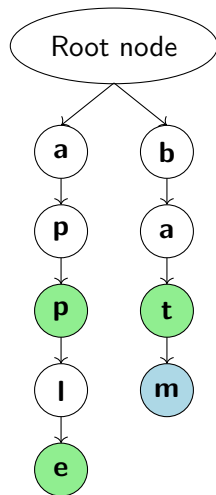
Insert "batman"



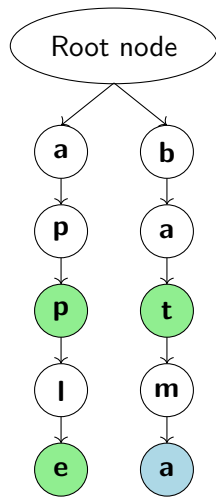
Insert "batman"



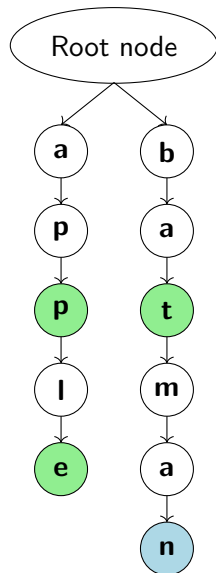
Insert "batman"



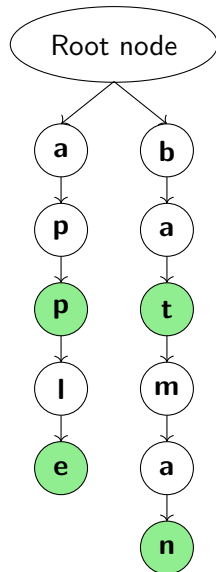
Insert "batman"



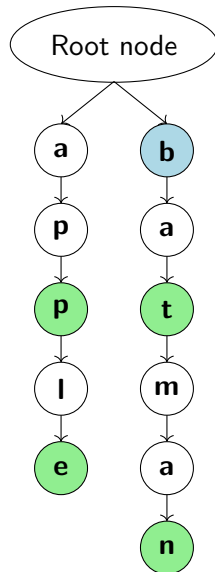
Insert "batman"



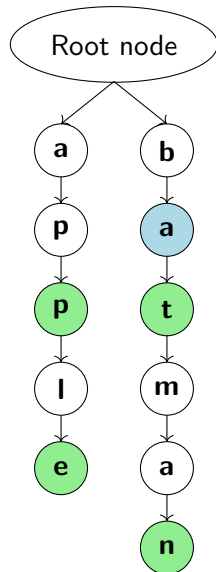
Insert "batman"



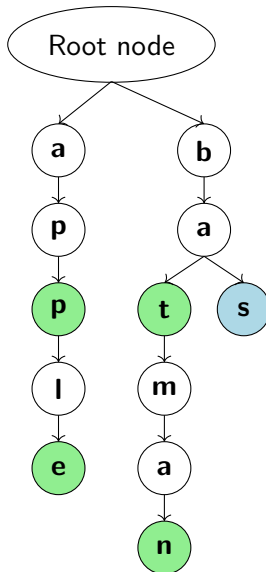
Insert "base"



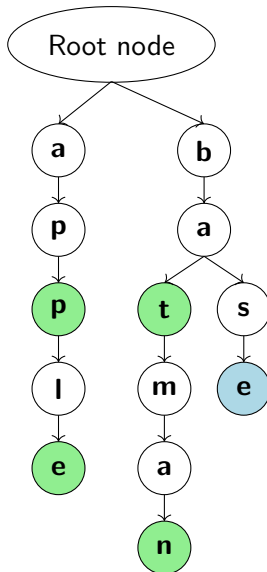
Insert "base"



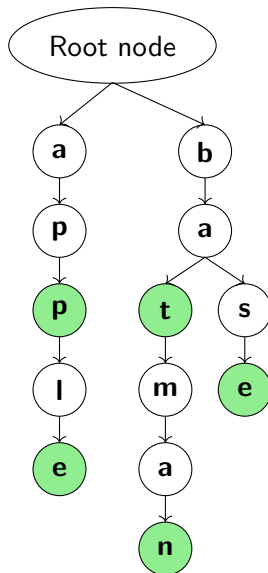
Insert "base"



Insert "base"



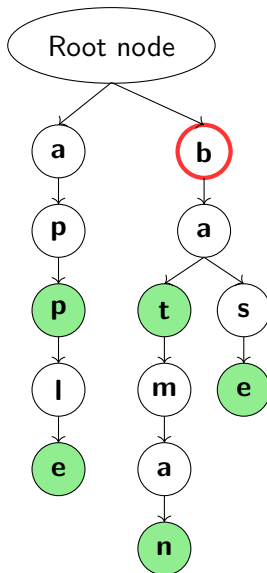
Insert "base"



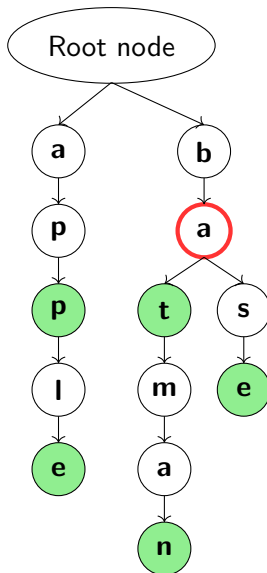
Search: O(L)

```
14 void insert(TrieNode* &proot, const string& word){
15     if (!proot) return;
16     TrieNode* temp = proot;
17
18     // Traverse each character of the word
19     for (char ch : word){
20         // If the character is not present, create a new TrieNode
21         if(temp->children.find(ch)==temp->children.end())
22             temp->children[ch] = new TrieNode;
23         temp = temp->children[ch];
24     }
25     // Mark the last node as the end of a word
26     temp->EndofWord=true;
27 }
```

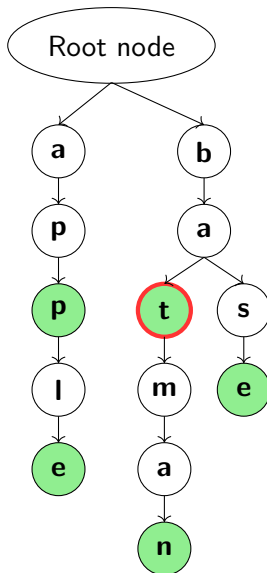
Search "bat"



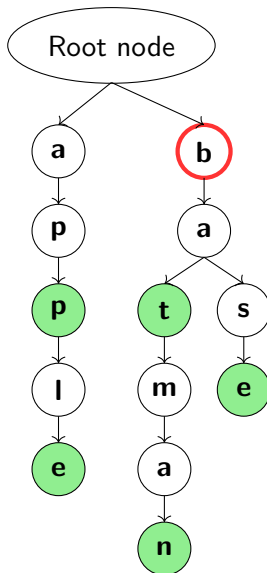
Search "bat"



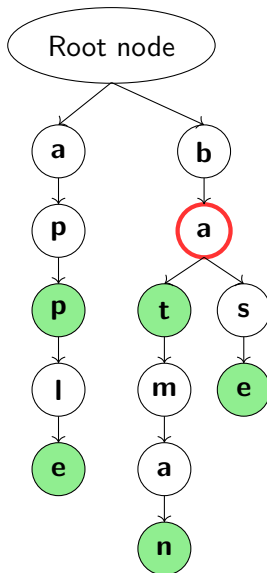
Search "bat": TRUE



Search "ba"



Search "ba": FALSE

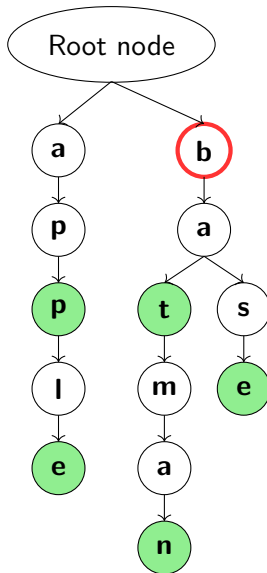


Delete: $O(L)$

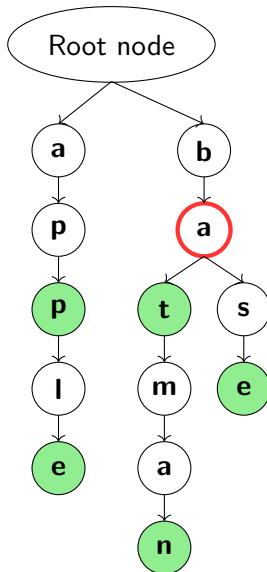
```
40 bool removeWord(TrieNode* proot, const string key, int depth = 0){
41     if (!proot) return false;
42
43     // Base case: if we reach the end of the word
44     if (depth==key.size()){
45         // If the word does not exist, return false
46         if (!proot->EndofWord) return false;
47         // Unmark this node as an end of a word
48         proot->EndofWord=false;
49
50         // If this node has no children, it can be deleted
51         if (proot->children.empty()){
52             return true;
53         }
54         return false;
55     }
56     // Recursively delete the next character in the word
57     if (removeWord(proot->children[key[depth]], key, depth + 1)){
58         delete proot->children[key[depth]];
59         proot->children.erase(key[depth]);
60
61         // If this node is not an end of another word and has no children, it
62         // can be deleted
63         return !proot->EndofWord && proot->children.empty();
64     }
65     return false;
66 }
```



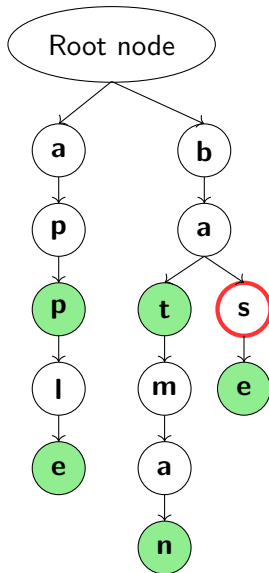
Delete "base"



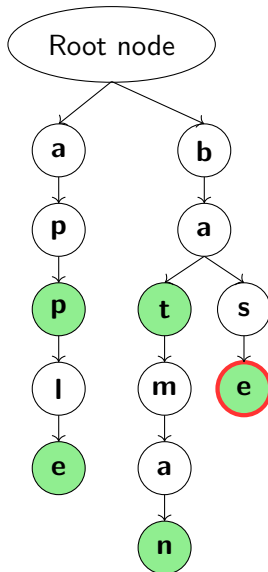
Delete "base"



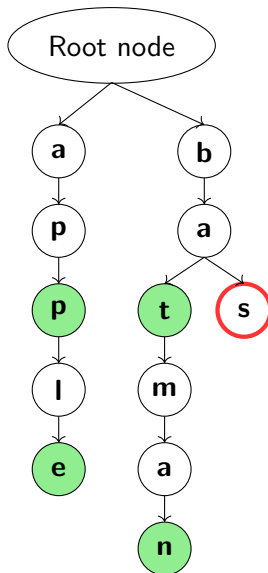
Delete "base"



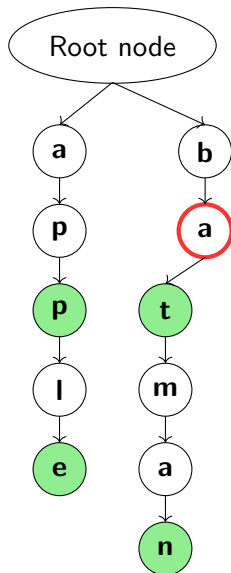
Delete "base"



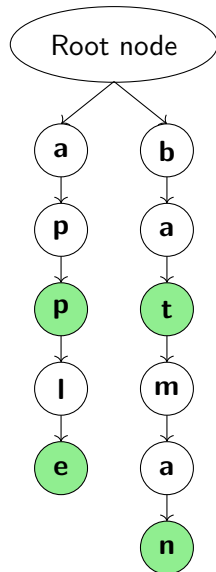
Delete "base"



Delete "base"



Insert "batman"



Space Complexity

$O(N * M)$, where N is the number of words and M is the average word length. Each character across all words requires a node (or a slot in a dictionary/array), causing the space to scale with the total number of characters stored.

Competitive Programming Applications

1) String Processing

- String Matching and Dictionary Problems

- Suffix Trie for Pattern Searching

2) Optimization and Searching

- Longest Prefix Matching Algorithm

- Counting Unique Substrings (Suffix Trie)

- XOR Maximization

3) Dynamic Programming and Optimization

- Dynamic Programming with Trie

 - Word Break Problem: Determines if a string can be segmented into valid words using DP + Trie.

 - String DP Problems: Optimization for finding shortest unique prefixes.



Practical Applications

1) Search and Retrieval

Search Engines and Autocomplete.

Google search and keyboards use Trie-based approaches for fast suggestions.
Helps in auto-correct and word prediction in text input.

Dictionary

2) Data Compression and Encoding

Compression and Encoding

Used in Huffman Coding to build prefix-free encoding schemes.
Some Lempel-Ziv compression algorithms use Tries for efficient substring matching.



Practical Applications

3) Networking and Security

Network Routing and IP Lookup

Trie-based structures optimize IP routing tables for efficient packet forwarding.
Used in firewall rule matching and domain name resolution.

4) Natural Language Processing (NLP)

Helps in word segmentation for text analysis.

Used in AI chatbots for fast word lookups and pattern recognition.



Advantages

- ★ Efficient for searching, inserting and deleting words in the time complexity $O(L)$, where L is the length of the word (faster than both the hash tables and the binary search tree in prefix matching).
- ★ Supports prefix-based operations efficiently, making it useful for autocomplete and dictionary implementations.
- ★ Can handle a large number of words without collisions, unlike hash tables.
- ★ Provides lexicographical sorting of stored words naturally.
- ★ Useful in applications such as IP routing, search engines, and natural language processing.
- ★ More space efficient compare to binary search tree.

Disadvantages

- ★ High memory consumption due to storing multiple nodes for each character.
- ★ Not space-efficient when storing a small number of words compared to hash tables.
- ★ Can be slower than hash tables for exact word searches since hashing has average $O(1)$ lookup time.
- ★ Requires extra implementation effort compared to simpler data structures.

1. Trie is also known as

- a. Prefix Tree
- b. Treap
- c. Binomial Tree
- d. 2-3 Tree

1. Trie is also known as

- a. **Prefix Tree**
- b. Treap
- c. Binomial Tree
- d. 2-3 Tree

What traversal over trie gives the lexicographical sorting of the set of the strings?

- a. Postorder
- b. Preorders
- c. Inorder
- d. Level order

What traversal over trie gives the lexicographical sorting of the set of the strings?

- a. Postorder
- b. **Preorders**
- c. Inorder
- d. Level order

Which of the following is not true?

- a. Trie requires less storage space than hashing
- b. Trie allows listing of all the words with same prefix
- c. Tries are collision free
- d. Trie is also known as prefix tree

Which of the following is not true?

- a. **Trie requires less storage space than hashing**
- b. Trie allows listing of all the words with same prefix
- c. Tries are collision free
- d. Trie is also known as prefix tree

What can be the maximum depth of the trie with n strings and m as the maximum string the length?

- a. $\log_2(n)$
- b. $\log_2(m)$
- c. n
- d. m



What can be the maximum depth of the trie with n strings and m as the maximum string length?

- a. $\log_2(n)$
- b. $\log_2(m)$
- c. n
- d. m

Which of the following is true about the trie?

- a. root is letter 'a'
- b. path from root to the leaf yields the string
- c. children of nodes are randomly ordered
- d. each node stores the associated keys

Which of the following is true about the trie?

- a. root is letter 'a'
- b. **path from root to the leaf yields the string**
- c. children of nodes are randomly ordered
- d. each node stores the associated keys